qual_summary.py 사용법 및 설명서

- 위치: `lab11/src/qual_summary.py`
- 목적: FASTQ 파일의 모든 read에 대해 평균 품질 점수(Phred score)를 구하고, 그 값들의 전체 평균을 출력한다.
- 필요한 것: Python 3 (표준 라이브러리 `sys`만 사용, 추가 설치 없음)

---

1. 빠른 사용법

`lab11` 폴더에서 다음과 같이 실행한다.

```
python3 src/qual_summary.py data/SRR1553607_1.fastq
```

형식:

```
python3 src/qual_summary.py <FASTQ 파일 경로>
```

실행 결과 예시

```
전체 read 평균 품질: 33.4497222958299
```

출력 항목	의미
전체 read 평균 품질	read마다 구한 평균 품질 점수를 다시 평균 낸 값

수업 데이터 두 파일의 결과는 다음과 같다. (실행 시간 약 1초)

파일	read 수	전체 read 평균 품질
`data/SRR1553607_1.fastq`	203,445	33.45
`data/SRR1553607_2.fastq`	203,445	30.83

역방향 read 파일(`_2`)의 품질이 조금 더 낮다. Illumina paired-end 시퀀싱에서는 두 번째 read의 품질이 낮게 나오는 경우가 흔하다.

---

2. 품질 점수 복습

FASTQ 파일은 **4줄이 read 1개**이고, 4번째 줄이 품질 점수이다.

```
@SRR1553607.1 1 length=101     <- 1줄: '@' + read 이름
GTTAGCGTTGTTGATCGCGACG...      <- 2줄: 염기서열
```

```
+SRR1553607.1 1 length=101     <- 3줄: '+' (구분자)
BB@FFFFFHHHHHJJJJJJJJJ...      <- 4줄: 품질 점수 (서열과 길이가 같음)
```

품질 줄의 글자 하나가 염기 하나의 품질을 나타낸다. 글자를 숫자로 바꾸는 규칙(**Phred+33**)은 다음과 같다.

```
품질 점수 Q = (글자의 ASCII 코드) - 33
```

글자	ASCII 코드 (ord)	품질 점수 Q
@	64	31
B	66	33
F	70	37
H	72	39
J	74	41

Q 값은 그 염기가 틀렸을 확률과 관계가 있다.

Q	염기가 틀렸을 확률	정확도
10	1/10	90%
20	1/100	99%
30	1/1,000	99.9%
40	1/10,000	99.99%

보통 **Q30 이상**이면 품질이 좋다고 본다. 수업 데이터의 평균(약 31~33)은 좋은 편이다.

---

3. 코드 설명

```
import sys

def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records

def mean_quality(qual):
    scores = []
```

```
    for ch in qual:
        scores.append(ord(ch) - 33)
    return sum(scores) / len(scores)

reads = read_fastq(sys.argv[1])

total = 0
for name, seq, qual in reads:
    q = mean_quality(qual)
    total = total + q

print("전체 read 평균 품질:", total / len(reads))
```

프로그램은 세 부분으로 나뉜다.

1. `read_fastq()` — 파일을 읽어 read 목록을 만든다.
2. `mean_quality()` — read 하나의 평균 품질을 계산한다.
3. 메인 부분 — 모든 read의 평균 품질을 더한 뒤 read 수로 나누어 출력한다.

3-1. `read_fastq(filename)` 함수

`parse_fastq.py`의 함수와 같다. (자세한 설명은 `doc/parse_fastq_usage.md` 참고)

코드	설명
`with open(filename) as f:`	파일을 열고, 블록이 끝나면 자동으로 닫는다
`lines = f.readlines()`	파일의 모든 줄을 리스트로 읽는다
`for i in range(0, len(lines), 4):`	4줄씩 건너뛰며 각 read의 첫 줄 위치를 가리킨다
`lines[i].rstrip()[1:]`	1줄: 줄바꿈 제거 후 맨 앞 `@`를 잘라낸다 → 이름
`lines[i + 1].rstrip()`	2줄: 염기서열
`lines[i + 3].rstrip()`	4줄: 품질 점수 문자열
`records.append((name, seq, qual))`	세 값을 튜플로 묶어 리스트에 추가

3-2. `mean_quality(qual)` 함수

품질 문자열 하나를 받아 평균 품질 점수(실수)를 돌려준다.

코드	설명
`scores = []`	각 글자의 점수를 담을 빈 리스트
`for ch in qual:`	문자열을 한 글자씩 꺼낸다
`ord(ch)`	글자의 ASCII 코드 숫자를 돌려준다. 예: `ord("J")` → 74
`ord(ch) - 33`	Phred+33 규칙으로 품질 점수를 구한다. 예: 74 - 33 = 41
`scores.append(...)`	구한 점수를 리스트에 추가

`sum(scores) / len(scores)`	점수의 합 ÷ 개수 = 평균

계산 예 (`"BF@J"`라는 짧은 품질 문자열):

```
B → 66 - 33 = 33
F → 70 - 33 = 37
@ → 64 - 33 = 31
J → 74 - 33 = 41
평균 = (33 + 37 + 31 + 41) / 4 = 35.5
```

수업 데이터 첫 번째 read(길이 101)의 평균 품질은 약 **37.44**이다.

3-3. 메인 부분

코드	설명
`reads = read_fastq(sys.argv[1])`	명령줄에서 받은 파일 경로로 모든 read를 읽는다
`total = 0`	평균 품질을 더해 갈 변수
`for name, seq, qual in reads:`	튜플 `(이름, 서열, 품질)`을 세 변수로 나누어 받는다 (**언패킹**)
`q = mean_quality(qual)`	이 read의 평균 품질
`total = total + q`	누적 합에 더한다
`total / len(reads)`	누적 합 ÷ read 수 = 전체 평균

`sys.argv`는 명령줄에 입력한 값들의 리스트이다.

- `sys.argv[0]` → `"src/qual_summary.py"` (스크립트 이름)
- `sys.argv[1]` → `"data/SRR1553607_1.fastq"` (읽을 파일)

---

4. "read 평균의 평균" 이라는 점

이 프로그램이 출력하는 값은 **read별 평균을 다시 평균 낸 값**이다. 모든 염기의 점수를 한꺼번에 평균 낸 값과는 read 길이가 서로 다를 때 달라진다.

예: read A(길이 2, 점수 40, 40)와 read B(길이 4, 점수 20, 20, 20, 20)

```
read 평균의 평균 = (40 + 20) / 2          = 30.0
전체 염기의 평균 = (40*2 + 20*4) / (2 + 4) = 26.7
```

수업 데이터는 모든 read 길이가 101로 같으므로 두 값이 같다.

---

5. 주의할 점과 한계

1. **파일 이름을 안 주면 오류**: `python3 src/qual_summary.py` 처럼 실행하면 `sys.argv[1]`이 없어서 `IndexError`가 난다.
2. **빈 파일이면 오류**: read가 0개이면 `total / len(reads)`에서 0으로 나누게 되어 `ZeroDivisionError`가 난다.
3. **품질 줄이 비어 있으면 오류**: `mean_quality()`에서 `len(scores)`가 0이 되어 `ZeroDivisionError`가 난다.
4. **줄 수가 4의 배수가 아니면 오류**: 잘린 파일이면 `lines[i + 3]`이 없어 `IndexError`가 날 수 있다.
5. **Phred+33만 가정**: 오래된 Illumina 데이터(Phred+64)를 넣으면 점수가 31만큼 높게 계산된다.
6. **메모리 사용**: `readlines()`와 `records` 리스트가 파일 전체를 메모리에 올린다. 수업 데이터는 괜찮지만 수 GB 크기의 FASTQ에서는 메모리가 부족할 수 있다.
7. **결과가 하나뿐**: 전체 평균 하나만 출력한다. read별 품질, 위치별 품질, Q30 이상 비율 등은 알 수 없다.

6. 수업 데이터 실행 명령

```
python3 src/qual_summary.py data/SRR1553607_1.fastq
python3 src/qual_summary.py data/SRR1553607_2.fastq
```